

Lab 6 - Terraform + Ansible + Docker + Kubernetes: EKS + ECR + Helm CI-CD Pipeline

Full Lab Guide, Configuration & Source Code

Terraform / Ansible / Docker / Kubernetes Lab Series

Contents

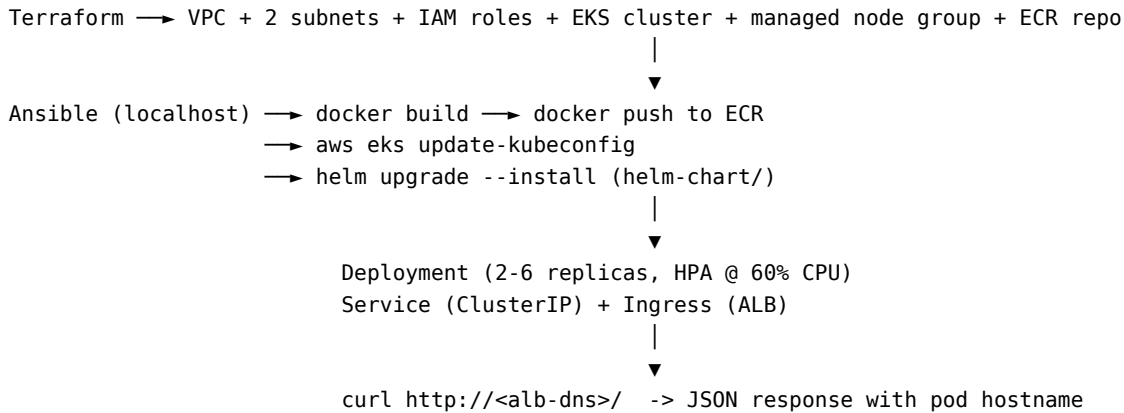
Use Case	1
Architecture	2
Directory Layout	2
Prerequisites	2
Step-by-Step Configuration & Execution	3
Step 1 — Provision EKS + ECR with Terraform	3
Step 2 — Point Helm's values at the real ECR repo	3
Step 3 — Run the Ansible build/push/deploy pipeline	3
Validation Steps	3
Cleanup	4
Key Concepts Reinforced	4

Appendix: Full Source Code	5
ansible/inventory.ini	5
ansible/playbook.yml	5
ansible/roles/build_push/tasks/main.yml	5
docker/Dockerfile	6
docker/app.py	6
docker/requirements.txt	7
helm-chart/Chart.yaml	7
helm-chart/templates/deployment.yaml	7
helm-chart/templates/hpa.yaml	8
helm-chart/templates/ingress.yaml	8
helm-chart/templates/service.yaml	9
helm-chart/values.yaml	9
k8s/load-generator.yaml	9
terraform/main.tf	10
terraform/outputs.tf	13
terraform/variables.tf	13

Use Case

A team wants a production-style pipeline: **Terraform** provisions a managed **EKS** cluster and an **ECR** repository; **Ansible** orchestrates building the **Docker** image, pushing it to ECR, and deploying it to the cluster via a **Helm** chart that includes autoscaling (HPA) and an ALB Ingress. This is the pattern most teams actually run in production — Terraform for durable cloud infra, Ansible/CI for the build-and-release step, Helm for application packaging.

Architecture



Directory Layout

```

lab6-tf-ansible-docker-k8s-eks-helm/
├── terraform/
│   ├── main.tf          # VPC, IAM roles, EKS cluster + node group, ECR repo
│   ├── variables.tf
│   └── outputs.tf       # cluster_name, cluster_endpoint, ecr_repository_url
├── docker/
│   ├── app.py           # Flask app returning version + pod hostname
│   ├── requirements.txt
│   └── Dockerfile
├── ansible/
│   ├── inventory.ini    # localhost (build/deploy orchestrator)
│   ├── playbook.yml
│   └── roles/build_push/tasks/main.yml # build, push, update-kubeconfig, helm upgrade
├── helm-chart/
│   ├── Chart.yaml
│   ├── values.yaml
│   └── templates/
│       ├── deployment.yaml
│       ├── service.yaml
│       ├── hpa.yaml
│       └── ingress.yaml
└── k8s/
    └── load-generator.yaml # busybox pod to drive CPU load for HPA validation
  
```

Prerequisites

- Terraform $\geq$ 1.5.0, AWS CLI v2 configured with EKS/IAM/ECR permissions
- Docker installed locally (or on the Ansible control node)
- kubectl, helm $\geq$ 3.x installed locally
- Ansible $\geq$ 2.14 + ansible-galaxy collection install community.docker
- **AWS Load Balancer Controller** installed in the cluster if you want the Ingress to actually provision an ALB (eksctl/Helm install — see AWS docs); otherwise leave ingress.enabled=false in values.yaml and use kubectl port-forward for validation instead

Step-by-Step Configuration & Execution

Step 1 — Provision EKS + ECR with Terraform

```
cd terraform
terraform init
terraform apply -var="cluster_name=lab6-eks-cluster"
```

This takes ~12-15 minutes (EKS control plane provisioning is slow — this is normal). Note the outputs:

```
terraform output cluster_name
terraform output ecr_repository_url
```

Step 2 — Point Helm's values at the real ECR repo

Edit helm-chart/values.yaml:

```
image:
  repository: "<ecr_repository_url from Step 1>"
  tag: "v1"
```

(The Ansible playbook also passes `--set image.repository=...` explicitly, so this edit is a convenience for direct helm install testing.)

Step 3 — Run the Ansible build/push/deploy pipeline

```
cd ../ansible
ansible-galaxy collection install community.docker
ansible-playbook playbook.yml \
  -e "aws_region=us-east-1" \
  -e "ecr_repo_url=$(cd ../terraform && terraform output -raw ecr_repository_url)" \
  -e "cluster_name=$(cd ../terraform && terraform output -raw cluster_name)" \
  -e "image_tag=v1"
```

This authenticates Docker to ECR, builds docker/Dockerfile, pushes `<ecr_repo_url>:v1`, updates local kubeconfig for the EKS cluster, and runs `helm upgrade --install lab6-app helm-chart/ --wait`.

Validation Steps

1. EKS nodes healthy

```
kubectl get nodes
# Expect: node_desired_size nodes, STATUS = Ready
```

2. ECR image pushed

```
aws ecr describe-images --repository-name lab6-demo-app --region us-east-1
```

3. Helm release deployed

```
helm status lab6-app
kubectl get deploy,svc,hpa,ingress
```

4. Pods running and serving traffic

```
kubectl port-forward svc/lab6-app-svc 8080:80 &
curl http://localhost:8080/
# {"message": "...", "version": "1.0.0", "hostname": "lab6-app-xxxxx"}
```

5. Rolling update / version bump validation

```
# bump APP_VERSION in docker/Dockerfile, then re-run Step 3 with image_tag=v2
kubectl rollout status deployment/lab6-app
curl http://localhost:8080/ | grep version # now shows 2.0.0 (or whatever you set)
```

6. Autoscaling (HPA) validation

```
kubectl apply -f ../k8s/load-generator.yaml
kubectl get hpa lab6-app-hpa --watch
# TARGETS column climbs toward/above 60%, REPLICAS increases from 2 toward maxReplicas
kubectl delete pod load-generator # stop the load once validated
```

7. Ingress reachable (if ALB controller installed)

```
kubectl get ingress lab6-app-ingress
ALB_DNS=$(kubectl get ingress lab6-app-ingress -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
curl -H "Host: lab6-demo.example.com" http://$ALB_DNS/
```

Cleanup

```
helm uninstall lab6-app
cd ../terraform
terraform destroy -var="cluster_name=lab6-eks-cluster"
```

Destroy the Helm release and any ALB Ingress **before** terraform destroy, otherwise the ALB controller-provisioned load balancer and its ENIs can block VPC/subnet deletion.

Key Concepts Reinforced

- Terraform manages durable, slow-changing infra (EKS control plane, IAM, VPC, ECR)
- Ansible orchestrates the fast-changing build/release cycle (build → push → deploy)
- Helm packages the Kubernetes application with parameterized `values.yaml`
- HPA + a synthetic load generator is the only reliable way to *prove* autoscaling works, not just that it's configured
- Ordering matters on teardown: app-layer objects (Ingress/ALB) before infra-layer (terraform destroy)

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/inventory.ini

```
[build_host]
localhost ansible_connection=local
```

ansible/playbook.yml

```
---
- name: Build, push, and deploy the containerized app to EKS via Helm
  hosts: build_host
  gather_facts: false

  vars:
    aws_region: "{{ aws_region | default('us-east-1') }}"
    ecr_repo_url: "{{ ecr_repo_url }}"           # from: terraform output ecr_repository_url
    cluster_name: "{{ cluster_name }}"          # from: terraform output cluster_name
    image_tag: "{{ image_tag | default('v1') }}"

  roles:
    - build_push
```

ansible/roles/build_push/tasks/main.yml

```
---
- name: Authenticate Docker to ECR
  shell: >
    aws ecr get-login-password --region {{ aws_region }}
    | docker login --username AWS --password-stdin {{ ecr_repo_url.split('/')[0] }}
  changed_when: false

- name: Build the Docker image
  community.docker.docker_image:
    name: "{{ ecr_repo_url }}"
    tag: "{{ image_tag }}"
    source: build
    build:
      path: "{{ playbook_dir }}/../docker"
    push: false
    force_source: true

- name: Push the image to ECR
  community.docker.docker_image:
    name: "{{ ecr_repo_url }}"
    tag: "{{ image_tag }}"
    source: local
    push: true
    repository: "{{ ecr_repo_url }}:{{ image_tag }}"
```

```

- name: Update local kubeconfig for the target EKS cluster
  command: aws eks update-kubeconfig --name {{ cluster_name }} --region {{ aws_region }}
  changed_when: false

- name: Deploy / upgrade the Helm release with the new image
  command: >
    helm upgrade --install lab6-app {{ playbook_dir }}/../helm-chart
    --set image.repository={{ ecr_repo_url }}
    --set image.tag={{ image_tag }}
    --wait --timeout 5m
  register: helm_result

- name: Show Helm output
  debug:
    var: helm_result.stdout_lines

```

docker/Dockerfile

```

FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV APP_VERSION=1.0.0
EXPOSE 8080
CMD ["python", "app.py"]

```

docker/app.py

```

from flask import Flask
import os
import socket

app = Flask(__name__)
VERSION = os.environ.get("APP_VERSION", "1.0.0")

@app.route("/")
def index():
    return {
        "message": "Lab 6: Terraform (EKS) + Ansible (build/push) + Docker + Kubernetes (Helm)",
        "version": VERSION,
        "hostname": socket.gethostname(),
    }

@app.route("/healthz")
def healthz():
    return {"status": "ok"}

```

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=8080)
```

docker/requirements.txt

```
flask==3.0.3
```

helm-chart/Chart.yaml

```
apiVersion: v2  
name: lab6-app  
description: Lab 6 demo app - Flask container deployed to EKS via Helm  
type: application  
version: 0.1.0  
appVersion: "1.0.0"
```

helm-chart/templates/deployment.yaml

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: {{ .Release.Name }}  
  labels:  
    app: {{ .Release.Name }}  
spec:  
  replicas: {{ .Values.replicaCount }}  
  selector:  
    matchLabels:  
      app: {{ .Release.Name }}  
  template:  
    metadata:  
      labels:  
        app: {{ .Release.Name }}  
    spec:  
      containers:  
        - name: app  
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"  
          imagePullPolicy: {{ .Values.image.pullPolicy }}  
          ports:  
            - containerPort: {{ .Values.service.targetPort }}  
          resources:  
            {{- toYaml .Values.resources | nindent 12 }}  
          readinessProbe:  
            httpGet:  
              path: /healthz  
              port: {{ .Values.service.targetPort }}  
            initialDelaySeconds: 5  
            periodSeconds: 10  
          livenessProbe:  
            httpGet:  
              path: /healthz  
              port: {{ .Values.service.targetPort }}
```

```
initialDelaySeconds: 15
periodSeconds: 20
```

helm-chart/templates/hpa.yaml

```
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ .Release.Name }}-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Release.Name }}
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
{{- end }}
```

helm-chart/templates/ingress.yaml

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ .Release.Name }}-ingress
  annotations:
    kubernetes.io/ingress.class: {{ .Values.ingress.className }}
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  rules:
    - host: {{ .Values.ingress.host }}
      http:
        paths:
          - path: {{ .Values.ingress.path }}
            pathType: Prefix
            backend:
              service:
                name: {{ .Release.Name }}-svc
                port:
                  number: {{ .Values.service.port }}
{{- end }}
```


helm-chart/templates/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-svc
spec:
  type: {{ .Values.service.type }}
  selector:
    app: {{ .Release.Name }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}
```

helm-chart/values.yaml

```
replicaCount: 2

image:
  repository: "REPLACE_WITH_ECR_REPO_URL"
  tag: "v1"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80
  targetPort: 8080

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 250m
    memory: 256Mi

autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
  targetCPUUtilizationPercentage: 60

ingress:
  enabled: true
  className: alb
  host: lab6-demo.example.com
  path: /
```

k8s/load-generator.yaml

```
apiVersion: v1
kind: Pod
```

```

metadata:
  name: load-generator
spec:
  containers:
  - name: busybox
    image: busybox
    command:
    - /bin/sh
    - -c
    - "while true; do wget -q -O- http://lab6-app-svc.default.svc.cluster.local; done"
  restartPolicy: Never

```

terraform/main.tf

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_availability_zones" "available" {
  state = "available"
}

# -----
# Networking (minimal VPC dedicated to the EKS cluster)
# -----

resource "aws_vpc" "eks_vpc" {
  cidr_block      = "10.10.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "lab6-eks-vpc" }
}

resource "aws_subnet" "eks_subnets" {
  count          = 2
  vpc_id         = aws_vpc.eks_vpc.id
  cidr_block     = "10.10.${count.index}.0/24"
  availability_zone = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name = "lab6-eks-subnet-${count.index}"
    "kubernetes.io/role/elb" = "1"
  }
}

```

```
resource "aws_internet_gateway" "eks_igw" {
  vpc_id = aws_vpc.eks_vpc.id
}

resource "aws_route_table" "eks_rt" {
  vpc_id = aws_vpc.eks_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.eks_igw.id
  }
}

resource "aws_route_table_association" "eks_rta" {
  count          = length(aws_subnet.eks_subnets)
  subnet_id      = aws_subnet.eks_subnets[count.index].id
  route_table_id = aws_route_table.eks_rt.id
}

# -----
# IAM roles required by EKS control plane and node group
# -----
resource "aws_iam_role" "eks_cluster_role" {
  name = "lab6-eks-cluster-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "eks.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  role          = aws_iam_role.eks_cluster_role.name
  policy_arn    = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

resource "aws_iam_role" "eks_node_role" {
  name = "lab6-eks-node-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "eks_worker_node_policy" {
  role          = aws_iam_role.eks_node_role.name
  policy_arn    = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}
```

```

resource "aws_iam_role_policy_attachment" "eks_cni_policy" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}

resource "aws_iam_role_policy_attachment" "eks_ecr_readonly" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

# -----
# EKS Cluster + managed node group
# -----
resource "aws_eks_cluster" "this" {
  name      = var.cluster_name
  role_arn  = aws_iam_role.eks_cluster_role.arn
  version   = var.kubernetes_version

  vpc_config {
    subnet_ids = aws_subnet.eks_subnets[*].id
  }

  depends_on = [aws_iam_role_policy_attachment.eks_cluster_policy]
}

resource "aws_eks_node_group" "default" {
  cluster_name    = aws_eks_cluster.this.name
  node_group_name = "lab6-default-ng"
  node_role_arn   = aws_iam_role.eks_node_role.arn
  subnet_ids      = aws_subnet.eks_subnets[*].id
  instance_types  = [var.node_instance_type]

  scaling_config {
    desired_size = var.node_desired_size
    min_size     = var.node_min_size
    max_size     = var.node_max_size
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_worker_node_policy,
    aws_iam_role_policy_attachment.eks_cni_policy,
    aws_iam_role_policy_attachment.eks_ecr_readonly,
  ]
}

# -----
# ECR repository for the application image built/pushed by Ansible
# -----
resource "aws_ecr_repository" "app" {
  name                = var.ecr_repo_name
  image_tag_mutability = "MUTABLE"
  force_delete        = true
}

```

terraform/outputs.tf

```
output "cluster_name" {
  value = aws_eks_cluster.this.name
}

output "cluster_endpoint" {
  value = aws_eks_cluster.this.endpoint
}

output "ecr_repository_url" {
  value = aws_ecr_repository.app.repository_url
}
```

terraform/variables.tf

```
variable "aws_region"           { default = "us-east-1" }
variable "cluster_name"        { default = "lab6-eks-cluster" }
variable "kubernetes_version"  { default = "1.30" }
variable "node_instance_type"  { default = "t3.medium" }
variable "node_desired_size"   { default = 2 }
variable "node_min_size"       { default = 2 }
variable "node_max_size"       { default = 4 }
variable "ecr_repo_name"       { default = "lab6-demo-app" }
```